TECHNICAL REFERENCE GUIDE • NLP SERIES

SELF-ATTENTION মেকানিজম

Word Embeddings থেকে Contextual Embeddings পর্যন্ত
— একটি সম্পূর্ণ বাংলা গাইড

বিষয়	ভাষা	ধরন
NLP & Transformers	বাংলা (Technical EN)	Conceptual + Code

সূচিপত্র — TABLE OF CONTENTS

০১ NLP-তে সবচেয়ে গুরুত্বপূর্ণ চ্যালেঞ্জ কী? §1
০২ One-Hot Encoding — প্রথম পদ্ধতি §2
০৩ Bag of Words — একটু উন্নতি §3
০৪ TF-IDF — আরও উন্নত পদ্ধতি §4
০৫ Word Embeddings — সেমান্টিক মিনিং ক্যাপচার §5
০৬ Word Embeddings-এর মূল সমস্যা — Static প্রকৃতি §6
০৭ Contextual Embedding — আদর্শ সমাধান §7
০৮ Self-Attention কী? — মূল সংজ্ঞা ও ধারণা §8
০৯ Self-Attention কীভাবে কাজ করে — Black Box দৃষ্টিকোণ §9
১০ Python কোড উদাহরণ — সম্পূর্ণ Implementation §10
১১ Transformers ও পরবর্তী পদক্ষেপ §11
১২ সারসংক্ষেপ ও মূল পয়েন্ট §12

অধ্যায় ০১

NLP-তে সবচেয়ে গুরুত্বপূর্ণ চ্যালেঞ্জ কী?

যেকোনো NLP অ্যাপ্লিকেশন তৈরি করতে হলে সবার আগে একটি মৌলিক প্রশ্নের উত্তর দিতে হবে — কম্পিউটার যেহেতু শুধু সংখ্যা বোঝে, তাহলে শব্দকে সংখ্যায় কীভাবে রূপান্তর করব?

মূল সমস্যাটি বোঝা

যেকোনো NLP (Natural Language Processing) অ্যাপ্লিকেশনের কথা ভাবো — Sentiment Analysis হোক, Named Entity Recognition হোক, বা Machine Translation। প্রতিটি ক্ষেত্রে ইনপুট হবে **শব্দ (Words)**।

কিন্তু কম্পিউটার শব্দ বোঝে না। কম্পিউটার বোঝে শুধু **সংখ্যা (Numbers)**। তাই সবচেয়ে গুরুত্বপূর্ণ প্রথম ধাপ হলো:

মূল চ্যালেঞ্জ

`Words → Numbers (Vectorization)`

এই প্রক্রিয়াকে NLP-র ভাষায় বলা হয় **Vectorization** — অর্থাৎ শব্দ বা টেক্সটকে Vector-এ রূপান্তর করা। বছরের পর বছর ধরে গবেষকরা এই সমস্যার বিভিন্ন সমাধান খুঁজে বের করেছেন।

> 💡 মনে রাখো
>
> NLP-র যেকোনো টাস্ক করতে হলে — সেটা Sentiment Analysis, Machine Translation, বা Named Entity Recognition যাই হোক — সবার আগে শব্দকে সংখ্যায় রূপান্তর করতে হবে। এটা ছাড়া কোনো অ্যাপ্লিকেশনই কাজ করবে না।

অধ্যায় ০২

ONE-HOT ENCODING — প্রথম পদ্ধতি

NLP-র সবচেয়ে প্রাচীন ও সরল পদ্ধতি — One-Hot Encoding। সহজ, কিন্তু বেশ সীমাবদ্ধ।

ONE-HOT ENCODING কীভাবে কাজ করে

ধরো তোমার কাছে দুটো sentence আছে:

> উদাহরণ SENTENCES
>
> Sentence 1: `mat` `cat` `mat`
>
> Sentence 2: `cat` `rat` `rat`

প্রথমে সব unique শব্দ বের করো:

Unique words: `mat` `cat` `rat` — মোট ৩টি

এরপর প্রতিটি শব্দকে এভাবে represent করো:

শব্দ	MAT	CAT	RAT	VECTOR
mat	1	0	0	[1, 0, 0]
cat	0	1	0	[0, 1, 0]
rat	0	0	1	[0, 0, 1]

তাহলে Sentence 1 (**mat cat mat**) হবে:

`[1,0,0] → [0,1,0] → [1,0,0]`

> ⚠️ সতর্কতা
>
> One-Hot Encoding অকার্যকর কারণ: (১) Vocabulary যত বড় হবে, Vector তত বড় হবে — লক্ষ লক্ষ শূন্য থাকবে। (২) শব্দের মধ্যে কোনো সম্পর্ক capture করা যায় না — "cat" আর "kitten" সম্পূর্ণ আলাদা vector পাবে অথচ এরা সমার্থক।

PYTHON কোড — ONE-HOT ENCODING

one_hot_encoding.py

```python
# One-Hot Encoding — সম্পূর্ণ উদাহরণ
import numpy as np

# Sentences
sentence1 = ["mat", "cat", "mat"]
sentence2 = ["cat", "rat", "rat"]

# সব unique শব্দ বের করো (Vocabulary)
all_words = sentence1 + sentence2
vocab = sorted(list(set(all_words)))
word_to_idx = {word: idx for idx, word in enumerate(vocab)}

print(f"Vocabulary: {vocab}")
print(f"Word to Index: {word_to_idx}")

# One-Hot Encode করার ফাংশন
def one_hot_encode(word, vocab_size, word_to_idx):
    vector = np.zeros(vocab_size, dtype=int)
    vector[word_to_idx[word]] = 1
    return vector

vocab_size = len(vocab)

# প্রতিটি শব্দের One-Hot Vector দেখো
for word in vocab:
    vec = one_hot_encode(word, vocab_size, word_to_idx)
    print(f"{word}: {vec}")

# Sentence-এর Representation
print("\nSentence 1 Representation:")
for word in sentence1:
    print(f"  {word} → {one_hot_encode(word, vocab_size, word_to_idx)}")

# Output:
# Vocabulary: ['cat', 'mat', 'rat']
# cat: [1 0 0]
# mat: [0 1 0]
# rat: [0 0 1]
# Sentence 1 Representation:
#   mat → [0 1 0]
#   cat → [1 0 0]
#   mat → [0 1 0]
```

অধ্যায় ০৩

BAG OF WORDS — একটু উন্নতি

One-Hot Encoding-এর একটু উন্নত রূপ হলো Bag of Words — এখানে শুধু শব্দ আছে কি নেই তা নয়, কতবার আছে তাও গোনা হয়।

BAG OF WORDS কীভাবে কাজ করে

One-Hot Encoding-এ শুধু বলা হতো শব্দটি sentence-এ **আছে কি নেই** (0 বা 1)। Bag of Words আরও এক ধাপ এগিয়ে — এটি বলে শব্দটি **কতবার** এসেছে।

SENTENCE	MAT (COUNT)	RAT (COUNT)	CAT (COUNT)	VECTOR
mat cat mat	2	0	1	[2, 0, 1]
cat rat rat	0	2	1	[0, 2, 1]

One-Hot-এর তুলনায় এটি একটু ভালো কারণ frequency-র তথ্য পাওয়া যায়। কিন্তু এখনও শব্দের **অর্থ বা সম্পর্ক** capture হয় না।

bag_of_words.py

```python
from collections import Counter

sentence1 = ["mat", "cat", "mat"]
sentence2 = ["cat", "rat", "rat"]

vocab = sorted(list(set(sentence1 + sentence2)))

def bag_of_words(sentence, vocab):
    count = Counter(sentence)
    return [count.get(word, 0) for word in vocab]

bow1 = bag_of_words(sentence1, vocab)
bow2 = bag_of_words(sentence2, vocab)

print(f"Vocab: {vocab}")
print(f"Sentence 1 BoW: {bow1}")  # [1, 2, 0]
print(f"Sentence 2 BoW: {bow2}")  # [1, 0, 2]
```

অধ্যায় ০৪

TF-IDF — আরও উন্নত পদ্ধতি

Bag of Words-এর আরও উন্নত সংস্করণ হলো TF-IDF। এটি কোনো শব্দ কতটা গুরুত্বপূর্ণ তা বিবেচনা করে।

TF-IDF কী এবং কেন?

Bag of Words-এর সমস্যা হলো — "the", "a", "is" এই সাধারণ শব্দগুলো অনেকবার আসলেও এরা কোনো বিশেষ মিনিং বহন করে না। TF-IDF এই সমস্যা সমাধান করে।

TF-IDF সূত্র

`TF-IDF(word) = TF(word) × IDF(word)`

`TF = Term Frequency | IDF = Inverse Document Frequency`

TF (Term Frequency): একটি document-এ একটি শব্দ কতবার এসেছে।
IDF (Inverse Document Frequency): সব document মিলিয়ে শব্দটি কতটা বিরল — বিরল হলে বেশি গুরুত্বপূর্ণ।

> 📌 গুরুত্বপূর্ণ
>
> TF-IDF, Bag of Words-এর চেয়ে ভালো, কিন্তু এখনও মৌলিক সমস্যা থেকেই যায় — শব্দের **semantic meaning** (অর্থ) এবং শব্দের মধ্যে **সম্পর্ক** capture হয় না। "King" আর "Queen" সম্পর্কিত হলেও তাদের vectors সম্পূর্ণ আলাদা থাকে।

অধ্যায় ০৫

WORD EMBEDDINGS — সেমান্টিক মিনিং ক্যাপচার

এখানেই NLP-তে বিপ্লব এলো। Word Embeddings শব্দের অর্থ এবং সম্পর্ককে সংখ্যায় ধরতে পারে।

WORD EMBEDDINGS কীভাবে তৈরি হয়

Word Embeddings-এর মূল শক্তি হলো এটি **Semantic Meaning** capture করতে পারে। মানে — শব্দটির অর্থ কী এবং সে সাধারণত কোন কোন প্রসঙ্গে ব্যবহার হয়, তা এনকোড করে।

এটি তৈরির প্রক্রিয়া:

বিশাল Training Data (Wikipedia সব Articles) → Neural Network Training → প্রতিটি শব্দের N-Dimension Vector

উদাহরণস্বরূপ, যদি আমরা **5-Dimension Vector** ব্যবহার করি, তাহলে "king" এবং "queen" শব্দ দুটো এভাবে represent হতে পারে:

শব্দ	DIM 1 (ROYALTY)	DIM 2 (ATHLETIC)	DIM 3 (HUMAN)	DIM 4	DIM 5
king	0.9 (হাই)	0.2 (লো)	0.8 (হাই)	0.6	0.2
queen	0.9 (হাই)	0.1 (লো)	0.9 (হাই)	0.5	0.3
cricketer	0.1 (লো)	0.9 (হাই)	0.8 (হাই)	0.2	0.7

লক্ষ্য করো — "king" এবং "queen" এর vector খুব কাছাকাছি (দুজনেই royal, human), কিন্তু "cricketer" সম্পূর্ণ আলাদা (athletic, non-royal)।

> 🔑 মূল ধারণা
>
> **সিমিলার শব্দ → সিমিলার Vector।** Word Embedding space-এ "king" এবং "queen" কাছাকাছি থাকবে, কিন্তু "cricketer" অনেক দূরে থাকবে। প্রতিটি Dimension কিছু একটা represent করে — Neural Network এটা নিজেই শিখে নেয়, আমাদের বলে দিতে হয় না।

word_embeddings_demo.py

```python
# Word Embeddings — gensim দিয়ে উদাহরণ
from gensim.models import Word2Vec
import numpy as np

# Sample Training Sentences
sentences = [
    ["king", "rules", "the", "kingdom"],
    ["queen", "rules", "the", "kingdom"],
    ["king", "and", "queen", "are", "royal"],
    ["cricketer", "plays", "cricket"],
    ["cricketer", "is", "an", "athlete"],
    ["apple", "is", "a", "delicious", "fruit"],
    ["apple", "launched", "a", "new", "phone"],
]

# Word2Vec Model Train করো
model = Word2Vec(
    sentences,
    vector_size=5,      # 5-Dimension Vector
    window=3,           # Context Window
    min_count=1,         # Minimum word frequency
    epochs=100
)

# শব্দের Vector দেখো
print("'king' এর Vector:", model.wv["king"])
print("'queen' এর Vector:", model.wv["queen"])
print("'cricketer' এর Vector:", model.wv["cricketer"])

# Similarity দেখো
similarity_king_queen = model.wv.similarity("king", "queen")
similarity_king_cricketer = model.wv.similarity("king", "cricketer")

print(f"\nKing ↔ Queen Similarity: {similarity_king_queen:.4f}")
print(f"King ↔ Cricketer Similarity: {similarity_king_cricketer:.4f}")
# King ↔ Queen এর similarity বেশি হবে!

# Apple এর Embedding দেখো (Static হওয়ার সমস্যা)
print(f"\n'apple' এর Static Embedding: {model.wv['apple']}")
# এই একটিই Vector দুটো অর্থের জন্য — fruit এবং tech company!
```

অধ্যায় ০৬

WORD EMBEDDINGS-এর মূল সমস্যা — STATIC প্রকৃতি

Word Embeddings অনেক শক্তিশালী, কিন্তু এর একটি মারাত্মক দুর্বলতা আছে — এটি "average meaning" capture করে, context-specific meaning নয়।

AVERAGE MEANING বনাম ACTUAL MEANING

Word Embedding Model একটি শব্দকে তার **Average Meaning** দিয়ে represent করে — মানে সেই শব্দটি training data-তে সাধারণত যেভাবে ব্যবহার হয়েছে তার গড়।

উদাহরণ: "Apple" শব্দটি নিয়ে ভাবো। Training Data-তে:

SENTENCE	APPLE এর অর্থ	প্রভাব
"An apple a day keeps the doctor away"	ফল (Fruit)	Taste dimension বাড়ে
"Apple is healthy"	ফল (Fruit)	Taste dimension আরও বাড়ে
"Apple is better than orange"	ফল (Fruit)	Taste dimension আরও বাড়ে
"Apple launched a new phone"	Tech Company	Technology dimension একটু বাড়ে

যদি 10,000 sentences-এ 9,000 বার "apple" ফল হিসেবে আসে এবং মাত্র 1,000 বার tech company হিসেবে, তাহলে final embedding হবে **Fruit-biased**।

> 🚨 মূল সমস্যা
>
> Word Embeddings **Static** — একবার train হলে সেটাই সব জায়গায় ব্যবহার হয়। "Apple launched a new phone" — এখানে "Apple" একটি tech company, কিন্তু Static Embedding এখানেও "Apple" কে একটি fruit হিসেবেই represent করবে কারণ সেই Embedding একবার তৈরি হয়েছে এবং সব context-এ একই থাকে।

PRACTICAL PROBLEM — MACHINE TRANSLATION উদাহরণ

ধরো তুমি একটি English → Hindi Machine Translation সিস্টেম বানাচ্ছো। একটি sentence পেলে:

> INPUT SENTENCE
>
> *"Apple launched a new phone while I was eating an orange."*

এখন এই sentence-টি process করতে প্রতিটি শব্দকে তার Embedding-এ রূপান্তর করতে হবে। কিন্তু সমস্যা কোথায়?

"Apple" → Static Embedding [9.3, 0.1, ...] "Fruit" biased → ❌ ভুল Context! Tech Company হওয়া উচিত

এই sentence-এ "Apple" স্পষ্টতই একটি **Tech Company** — কারণ "launched", "phone" এই শব্দগুলো আছে। কিন্তু Static Embedding এই context বুঝতে পারে না। "orange" দেখে আরও confused হতে পারে — মনে করতে পারে "apple" ফল হিসেবেই ব্যবহার হচ্ছে!

অধ্যায় ০৭

CONTEXTUAL EMBEDDING — আদর্শ সমাধান

Static Embedding-এর বিপরীতে আমাদের দরকার Contextual Embedding — যা context দেখে dynamically পরিবর্তিত হয়।

CONTEXTUAL EMBEDDING কী হওয়া উচিত

আদর্শ সমাধান হবে এমন একটি Embedding system যেখানে একটি শব্দের vector তার **surrounding context** দেখে dynamically adjust হবে।

"Apple launched a new phone while I was eating an orange" এই sentence-এ:

> ✅ CONTEXTUAL EMBEDDING কীভাবে কাজ করবে
>
> ১. "launched" এবং "phone" দেখে বুঝবে — Apple মানে Tech Company → Technology dimension বাড়াবে, Fruit/Taste dimension কমাবে।
>
> ২. "eating an orange" দেখে confused হবে না — কারণ "orange" এখানে "apple (fruit)" এর সাথে সম্পর্কিত নয়, এটি সম্পূর্ণ আলাদা clause-এ আছে।
>
> ৩. এই Embedding এতটাই smart হবে যে সে বুঝবে — এই sentence-এ "Apple" এবং "orange" সম্পর্কিত নয়।

তুলনামূলক পার্থক্য:

বৈশিষ্ট্য	STATIC WORD EMBEDDING	CONTEXTUAL EMBEDDING
Training	একবার train হয়	প্রতিটি sentence-এ dynamically তৈরি হয়
"Apple" vector	সব জায়গায় একই	Context অনুযায়ী ভিন্ন
Context awareness	নেই	আছে — surrounding words বিবেচনা করে
NLP Performance	সীমিত	অনেক বেশি ভালো

অধ্যায় ০৮

SELF-ATTENTION কী? — মূল সংজ্ঞা ও ধারণা

Contextual Embedding তৈরির সমস্যা সমাধান করার জন্যই এলো Self-Attention। এটাই Transformer-এর হৃদয়।

SELF-ATTENTION — সংক্ষিপ্ত সংজ্ঞা

Self-Attention হলো এমন একটি mechanism যা Static Embedding-কে input হিসেবে নিয়ে **Smart Contextual Embedding** output করে।

ONE-LINE DEFINITION

`Self-Attention = Static Embeddings → [Magic Box] → Contextual Embeddings`

এটাকে একটি **function** হিসেবে ভাবো। তুমি পুরো sentence-এর প্রতিটি শব্দের Static Embedding ভেতরে পাঠাও, এবং বের হয়ে আসে context-aware Contextual Embedding।

Apple Embedding | launched Embedding | phone Embedding | orange Embedding

↓

Self-Attention Mechanism (Internal Calculations)

↓

Apple Contextual | launched Contextual | phone Contextual | orange Contextual

লক্ষ্য করো — প্রতিটি input-এর জন্য একটি output Embedding বের হয় (**For every input, one output**)। কিন্তু এই output Embedding-গুলো অনেক স্মার্ট — এরা জানে যে "Apple" এই sentence-এ Tech Company হিসেবে ব্যবহার হচ্ছে।

SELF-ATTENTION — সারাংশ

- → Self-Attention একটি mechanism যা Transformer Architecture-এর মূল ভিত্তি।
- → এটি Static Word Embedding নিয়ে Dynamic Contextual Embedding তৈরি করে।
- → একটি শব্দ কোন context-এ ব্যবহার হচ্ছে তা বুঝে সেই অনুযায়ী তার representation adjust করে।
- → Self-Attention বুঝতে পারলে Transformer বোঝা অনেক সহজ হয়ে যায়।
- → Transformer বুঝলে LLMs, Generative AI সবকিছুতে দক্ষতা অর্জন করা যায়।

অধ্যায় ০৯

SELF-ATTENTION কীভাবে কাজ করে — BLACK BOX দৃষ্টিকোণ

এই অধ্যায়ে আমরা Self-Attention Mechanism-এর ভেতরে কী হচ্ছে তার conceptual ধারণা নেবো।

SENTENCE-এর প্রতিটি শব্দ একে অপরকে "দেখে"

Self-Attention-এর মূল ধারণাটা অনেকটা এরকম: একটি sentence-এর প্রতিটি শব্দ অন্য **সব শব্দকে** দেখে এবং বোঝার চেষ্টা করে যে তাদের মধ্যে কতটা সম্পর্ক আছে।

উদাহরণ: "Apple launched a new phone while I was eating an orange"

> ATTENTION PROCESS — "APPLE" শব্দের দৃষ্টিকোণ থেকে
>
> ❶ "Apple" → "launched" দেখে: বেশি attention → এটি একটি action, tech company-র কাজ
>
> ❷ "Apple" → "phone" দেখে: বেশি attention → tech product, তাই Apple = tech company
>
> ❸ "Apple" → "eating" দেখে: কম attention → খাওয়ার সাথে Apple-এর সম্পর্ক নেই এখানে
>
> ❹ "Apple" → "orange" দেখে: কম attention → ভিন্ন clause, সম্পর্ক নেই

এই attention scores-এর উপর ভিত্তি করে "Apple"-এর নতুন Contextual Embedding তৈরি হয় — যেখানে "launched" এবং "phone"-এর influence বেশি থাকে।

পরবর্তী ভিডিও — QUERY, KEY, VALUE VECTORS

Self-Attention-এর ভেতরে মূলত তিনটি গুরুত্বপূর্ণ concept আছে যা পরবর্তীতে বিস্তারিত আলোচনা হবে:

> আসন্ন TOPICS (NEXT VIDEOS)
>
> ① **Query Vector (Q)** — "আমি কী খুঁজছি?"
>
> ② **Key Vector (K)** — "আমার কাছে কী আছে?"
>
> ③ **Value Vector (V)** — "আমি কতটুকু দেবো?"
>
> ④ Attention Score Calculation
>
> ⑤ Softmax এবং Weighted Sum
>
> ⑥ Multi-Head Attention

Self-Attention মূলত এই Q, K, V vectors ব্যবহার করে প্রতিটি শব্দের জন্য Contextual Embedding তৈরি করে। এটি বিস্তারিত পরের পর্বে আলোচিত হবে।

অধ্যায় ১০

PYTHON কোড উদাহরণ — সম্পূর্ণ IMPLEMENTATION

এখন পর্যন্ত যা শিখেছি তার সবকিছু Python কোডে দেখা যাক — One-Hot থেকে শুরু করে Simplified Self-Attention পর্যন্ত।

সব VECTORIZATION METHOD তুলনা

vectorization_comparison.py

```python
"""
সব Vectorization পদ্ধতির তুলনা
One-Hot → Bag of Words → TF-IDF → Word Embeddings
"""
import numpy as np
from collections import Counter
import math

# ─── ১. One-Hot Encoding ───
def one_hot_encode(sentence, vocab):
    result = []
    for word in sentence:
        vec = [1 if w == word else 0 for w in vocab]
        result.append(vec)
    return result

# ─── ২. Bag of Words ───
def bag_of_words(sentence, vocab):
    count = Counter(sentence)
    return [count.get(w, 0) for w in vocab]

# ─── ৩. TF-IDF ───
def compute_tf_idf(sentences, vocab):
    N = len(sentences)
    results = []
    for sentence in sentences:
        tf_idf_vec = []
        word_count = Counter(sentence)
        for word in vocab:
            # TF: এই sentence-এ word-টি কতবার আছে
            tf = word_count.get(word, 0) / len(sentence)
            # DF: কতটি sentence-এ word-টি আছে
            df = sum(1 for s in sentences if word in s)
            # IDF: log(N/DF)
            idf = math.log(N / (df + 1))  # +1 to avoid division by zero
            tf_idf_vec.append(round(tf * idf, 4))
        results.append(tf_idf_vec)
    return results

# ─── Main ───
sentence1 = ["mat", "cat", "mat"]
sentence2 = ["cat", "rat", "rat"]
vocab = sorted(list(set(sentence1 + sentence2)))

print("=== One-Hot Encoding ===")
ohe = one_hot_encode(sentence1, vocab)
for word, vec in zip(sentence1, ohe):
    print(f"  {word}: {vec}")

print("\n=== Bag of Words ===")
print(f"  Sentence 1: {bag_of_words(sentence1, vocab)}")
print(f"  Sentence 2: {bag_of_words(sentence2, vocab)}")

print("\n=== TF-IDF ===")
tfidf = compute_tf_idf([sentence1, sentence2], vocab)
for i, vec in enumerate(tfidf):
    print(f"  Sentence {i+1}: {vec}")
```

SIMPLIFIED SELF-ATTENTION — NUMPY IMPLEMENTATION

self_attention_numpy.py

```python
"""
Simplified Self-Attention Implementation — NumPy দিয়ে
এটি Self-Attention-এর মূল concept বোঝার জন্য
(Full implementation-এ Q, K, V matrices থাকে)
"""
import numpy as np

def softmax(x):
    """Softmax function — scores কে probabilities-এ convert করে"""
    e_x = np.exp(x - np.max(x))  # numerical stability
    return e_x / e_x.sum(axis=-1, keepdims=True)

def simplified_self_attention(embeddings):
    """
    Simplified Self-Attention

    Input: embeddings — shape (seq_len, embed_dim)
           মানে প্রতিটি শব্দের Static Embedding

    Output: contextual_embeddings — shape (seq_len, embed_dim)
            মানে প্রতিটি শব্দের Contextual Embedding
    """
    seq_len, embed_dim = embeddings.shape

    # ① Attention Scores: প্রতিটি শব্দ অন্য সব শব্দের সাথে কতটা সম্পর্কিত
    # Dot Product ব্যবহার করে similarity measure করা হচ্ছে
    attention_scores = np.dot(embeddings, embeddings.T)
    print(f"Attention Scores (raw):\n{np.round(attention_scores, 2)}\n")

    # ② Scaling: embed_dim-এর square root দিয়ে ভাগ (gradient stability)
    scaled_scores = attention_scores / np.sqrt(embed_dim)

    # ③ Softmax: scores কে probabilities-এ convert করো
    # প্রতিটি row-এর sum = 1.0
    attention_weights = softmax(scaled_scores)
    print(f"Attention Weights (after softmax):\n{np.round(attention_weights, 3)}\n")

    # ④ Weighted Sum: attention weights দিয়ে embeddings-এর weighted average
    contextual_embeddings = np.dot(attention_weights, embeddings)

    return contextual_embeddings, attention_weights


# ─── উদাহরণ: "Apple launched phone" ───
np.random.seed(42)

# Simulate করা Static Embeddings (normally এগুলো Word2Vec/GloVe থেকে আসে)
# প্রতিটি শব্দের 4-dimension static embedding
apple_emb    = np.array([0.9, 0.1, 0.5, 0.3])  # fruit-biased
launched_emb = np.array([0.2, 0.8, 0.9, 0.6])  # action word
phone_emb    = np.array([0.1, 0.9, 0.8, 0.7])  # tech product
orange_emb   = np.array([0.8, 0.1, 0.2, 0.1])  # fruit (different context)

# Sentence: "Apple launched phone orange"
sentence_embeddings = np.array([apple_emb, launched_emb, phone_emb, orange_emb])
words = ["Apple", "launched", "phone", "orange"]

print("=== Static Embeddings (Input) ===")
for word, emb in zip(words, sentence_embeddings):
    print(f"  {word:10}: {emb}")

print("\n=== Self-Attention Processing ===")
contextual_embs, attn_weights = simplified_self_attention(sentence_embeddings)

print("=== Contextual Embeddings (Output) ===")
for word, emb in zip(words, contextual_embs):
    print(f"  {word:10}: {np.round(emb, 3)}")

# Apple-এর attention weights দেখো
# launched এবং phone-এর প্রতি attention বেশি হওয়া উচিত
print(f"\n'Apple' এর Attention Weights:")
for word, weight in zip(words, attn_weights[0]):
    bar = "█" * int(weight * 20)
    print(f"  → {word:10}: {weight:.3f} {bar}")
```

PYTORCH দিয়ে SELF-ATTENTION

self_attention_pytorch.py

```python
"""
PyTorch দিয়ে Self-Attention — Production-ready approach
এখানে Q, K, V (Query, Key, Value) matrices ব্যবহার করা হয়
"""
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class SelfAttention(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.embed_dim = embed_dim

        # Q, K, V Linear Projections
        # Static Embedding থেকে Q, K, V তৈরি করে
        self.W_q = nn.Linear(embed_dim, embed_dim)
        self.W_k = nn.Linear(embed_dim, embed_dim)
        self.W_v = nn.Linear(embed_dim, embed_dim)

        # Output Projection
        self.W_o = nn.Linear(embed_dim, embed_dim)

    def forward(self, x):
        """
        x: (batch_size, seq_len, embed_dim) — Static Embeddings
        return: (batch_size, seq_len, embed_dim) — Contextual Embeddings
        """
        # ① Query, Key, Value তৈরি করো
        Q = self.W_q(x)   # "আমি কী খুঁজছি?"
        K = self.W_k(x)   # "আমার কাছে কী আছে?"
        V = self.W_v(x)   # "আমি কতটুকু contribute করবো?"

        # ② Attention Scores: Q × K^T / √d_k
        scale = math.sqrt(self.embed_dim)
        scores = torch.bmm(Q, K.transpose(1, 2)) / scale

        # ③ Softmax — scores কে weights-এ convert
        attention_weights = F.softmax(scores, dim=-1)

        # ④ Weighted Sum of Values
        context = torch.bmm(attention_weights, V)

        # ⑤ Output Projection
        output = self.W_o(context)

        return output, attention_weights


# ─── Demo ───
batch_size = 1
seq_len = 4       # "Apple launched phone orange" = 4 শব্দ
embed_dim = 8      # 8-dimension embedding

# Random Static Embeddings (বাস্তবে এটি Word2Vec/GloVe থেকে আসবে)
torch.manual_seed(42)
static_embeddings = torch.randn(batch_size, seq_len, embed_dim)

# Self-Attention Layer
self_attn = SelfAttention(embed_dim)

# Forward Pass
contextual_embeddings, attn_weights = self_attn(static_embeddings)

print(f"Input Shape (Static Embeddings): {static_embeddings.shape}")
print(f"Output Shape (Contextual Embeddings): {contextual_embeddings.shape}")
print(f"Attention Weights Shape: {attn_weights.shape}")
print(f"\nAttention Weights (Word 0 'Apple' → others):")
print(attn_weights[0, 0].detach().numpy().round(3))

print("\n✅ Self-Attention সফলভাবে Contextual Embeddings তৈরি করেছে!")
```

STATIC বনাম CONTEXTUAL EMBEDDING — দৃশ্যমান পার্থক্য

static_vs_contextual.py

```python
"""
Static Embedding vs Contextual Embedding — পার্থক্য দেখা
একই শব্দ "apple" দুটো ভিন্ন context-এ কতটা আলাদা
"""
import torch
import numpy as np

# দুটো sentence যেখানে "apple" ভিন্ন অর্থে আছে
sentence_a = "Apple launched new phone"    # Tech Company
sentence_b = "Apple is healthy fruit"       # Fruit

# Static Embedding: "apple" সব জায়গায় একই
apple_static = np.array([0.5, 0.3, 0.7, 0.2])

print("=== Static Embeddings ===")
print(f"Sentence A-তে 'Apple': {apple_static}")
print(f"Sentence B-তে 'Apple': {apple_static}")
print(f"পার্থক্য: {np.linalg.norm(apple_static - apple_static):.4f} (শূন্য!)")

# Contextual Embedding: context দেখে ভিন্ন হয়
# (এটি simulate করা — বাস্তবে BERT/Transformer থেকে আসবে)
apple_in_tech_context  = np.array([0.1, 0.9, 0.8, 0.7])  # tech-oriented
apple_in_food_context  = np.array([0.9, 0.1, 0.2, 0.3])  # food-oriented

print("\n=== Contextual Embeddings (Self-Attention পরে) ===")
print(f"Sentence A-তে 'Apple' (tech): {apple_in_tech_context}")
print(f"Sentence B-তে 'Apple' (food): {apple_in_food_context}")

difference = np.linalg.norm(apple_in_tech_context - apple_in_food_context)
print(f"পার্থক্য: {difference:.4f} (শূন্য নয়! Context ধরা পড়েছে)")

# Cosine Similarity দেখো
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

sim_static = cosine_similarity(apple_static, apple_static)
sim_contextual = cosine_similarity(apple_in_tech_context, apple_in_food_context)

print(f"\nStatic 'apple' similarity (same everywhere): {sim_static:.4f}")
print(f"Contextual 'apple' similarity (tech vs food): {sim_contextual:.4f}")
print("\n✅ Self-Attention context capture করতে পারে!")
```

অধ্যায় ১১

TRANSFORMERS ও পরবর্তী পদক্ষেপ

Self-Attention বোঝা গেলে Transformer বোঝা অনেক সহজ। Transformer-ই আজকের LLM ও Generative AI-এর ভিত্তি।

SELF-ATTENTION → TRANSFORMER → LLMS

এই series-এর মূল লক্ষ্য হলো একটি সুস্পষ্ট learning path তৈরি করা:

Part 1 (এই ভিডিও) → Self-Attention কী সেটা বোঝা
Part 2 → Self-Attention কীভাবে কাজ করে (Q, K, V)
Part 3 → Behind-the-scenes geometry ও mathematics
পরবর্তী Chapters → Full Transformer → LLMs → Generative AI

TRANSFORMER ARCHITECTURE-এ SELF-ATTENTION

Transformer Architecture-এ Self-Attention একটি central role পালন করে। Transformer মূলত:

1. **Self-Attention Layer** — প্রতিটি শব্দ পুরো sequence দেখে Contextual Embedding তৈরি করে
2. **Multi-Head Attention** — একই সাথে অনেকগুলো attention pattern শেখে (ভিন্ন ভিন্ন relationship capture করার জন্য)
3. **Feed-Forward Network** — প্রতিটি position-এ independently apply হয়
4. **Layer Normalization** — Training stability নিশ্চিত করে
5. **Residual Connections** — Deep network-এ gradient flow সহজ করে

GPT, BERT, T5, Claude — সবই মূলত Transformer-এর উপর built। তাই Self-Attention বোঝাটা আজকের AI-এর সব কিছু বোঝার চাবিকাঠি।

> 🎯 পরবর্তী ভিডিওতে আসছে
>
> **Query (Q), Key (K), Value (V) Vectors** কী এবং কীভাবে এগুলো Self-Attention-এ ব্যবহার হয় — এটা পরের পর্বে বিস্তারিত আলোচনা হবে। এই তিনটি concept বুঝলেই Self-Attention-এর পুরো mechanism স্পষ্ট হয়ে যাবে।

অধ্যায় ১২

সারসংক্ষেপ ও মূল পয়েন্ট

এই গাইডে যা শিখলাম তার একটি সুসংহত সারসংক্ষেপ।

সম্পূর্ণ LEARNING JOURNEY

পদ্ধতি	বৈশিষ্ট্য	সমস্যা	যুগ
One-Hot Encoding	শব্দ আছে কি নেই (0/1)	Sparse, no meaning	প্রাচীন
Bag of Words	Word frequency count	No semantic meaning	প্রাচীন
TF-IDF	Weighted frequency	No semantic meaning	Traditional
Word Embeddings	Semantic meaning capture	Static — context-blind	Modern (2013+)
Self-Attention	Dynamic Contextual Embedding	—	State-of-the-Art

🔑 KEY TAKEAWAYS

① NLP-র সবচেয়ে মৌলিক চ্যালেঞ্জ হলো Words → Numbers (Vectorization)।
② Word Embeddings semantic meaning capture করতে পারে, কিন্তু এগুলো Static — একবার train হলে সব context-এ একই থাকে।
③ Word Embedding average meaning ধরে — training data-তে একটি শব্দ বেশিরভাগ সময় যে অর্থে ব্যবহার হয়েছে সেটাই তার embedding-এ প্রাধান্য পায়।
④ "Apple" এর উদাহরণ — fruit হিসেবে বেশি data থাকলে tech company context-এও fruit embedding পাবে। এটাই Static Embedding-এর মূল সমস্যা।
⑤ Self-Attention একটি mechanism যা Static Embeddings নিয়ে Smart Contextual Embeddings তৈরি করে — প্রতিটি শব্দের embedding তার surrounding context দেখে dynamically adjust হয়।
⑥ Self-Attention বোঝা = Transformer বোঝা = LLMs/Generative AI বোঝার ভিত্তি।
⑦ পরবর্তী পদক্ষেপ: Query (Q), Key (K), Value (V) Vectors শেখা — এগুলোই Self-Attention-এর ভেতরের mechanism।

> 📖 ONE-LINE SUMMARY
>
> *"Self-Attention is a mechanism that takes static embeddings as input and generates smart contextual embeddings as output — making it possible for models to understand words in their proper context."*